

ISTANBUL TECHNICAL UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

BLG 222E
COMPUTER ORGANIZATION
PROJECT 3 REPORT

CRN : 23727

LECTURER : Doç. Dr. Gökhan İnce

GROUP MEMBERS:

150230089 : Hüseyin Teke

150230040 : Aysun Dilara Doğan

SPRING 2026

Contents

1	INTRODUCTION	1
1.1	Task Distribution	2
2	MATERIALS AND METHODS	2
2.1	Data Path and Control Unit Architecture	2
2.2	Instruction Execution Phases and RTL Analysis	3
2.2.1	Instruction Fetch Cycle (2 Cycles)	4
2.2.2	Arithmetic, Logic, and Immediate Operations (3-5 Cycles)	4
2.2.3	Memory and Stack Operations (6-10 Cycles)	4
2.3	Detailed ROM Content and Line-by-Line Mapping	5
2.4	Source Code Repository	7
3	RESULTS	7
3.1	Comprehensive Instruction Cycle Analysis	7
3.2	Array Maximum Finder Verification	7
3.3	Factorial Calculation Verification and Waveform Analysis	8
4	DISCUSSION	11
5	CONCLUSION	12

1 INTRODUCTION

The primary objective of this project is the modular design and implementation of a complete hardwired control unit for a custom 16-bit computer architecture. Building upon the structural foundations laid in previous phases of the course, this project seamlessly integrates the Arithmetic Logic Unit (ALU), the General-Purpose Register File (RF), and the Address Register File (ARF) into a fully functional, autonomous Central Processing Unit (CPU).

In modern computer organization, the shift from merely designing datapath components to orchestrating them requires a robust control mechanism. To achieve this, the designed control unit employs a specialized Finite State Machine (FSM) mapped directly to a 12-bit ring counter. This FSM is responsible for generating the exact sequence of micro-operations needed to fetch, decode, and execute instructions. One of the primary architectural constraints addressed in this project is the Von Neumann bottleneck caused by a memory data width mismatch. Because the Data Memory Unit (RAM) has an 8-bit output bus, the control unit must dynamically manage fetching 16-bit instructions over multiple clock cycles (T_1 and T_2) using a little-endian format before initiating the execution phase at T_4 .

The implemented Instruction Set Architecture (ISA) accommodates an extensive suite of commands classified into two broad categories: address-referenced instructions (incorporating 8-bit immediate values or memory offsets) and non-address-referenced instructions (direct register-to-register manipulations). This encompasses direct/indirect memory addressing (LDA, STA), stack manipulations (PSH, POP), subroutine control (CALL, RET), indexed addressing (LDT, STT), and complex arithmetic logic.

The ultimate goal of this phase is to demonstrate the processor's capability to autonomously execute pre-compiled, machine-level algorithms. To rigorously verify the hardware, the CPU is tasked with executing two sophisticated programs directly from ROM: an Array Maximum Finder with pointer-based iteration, and a mathematical Factorial Calculator utilizing subroutine calls and conditional branching.

1.1 Task Distribution

Member	Responsibilities
Hüseyin Teke	Design of the Hardwired Control Unit FSM, implementation of the 12-bit ring counter logic, Verilog coding of the Instruction Decoder, and Datapath routing within CPUSystem.
Aysun Dilara Doğan	Manual translation of the Simple Program into hex codes, organization of memory initialization files and waveform analysis.

Table 1: Project task distribution among group members.

2 MATERIALS AND METHODS

2.1 Data Path and Control Unit Architecture

The designed processor relies on a hardwired control unit governed by a one-hot encoded Finite State Machine (FSM).

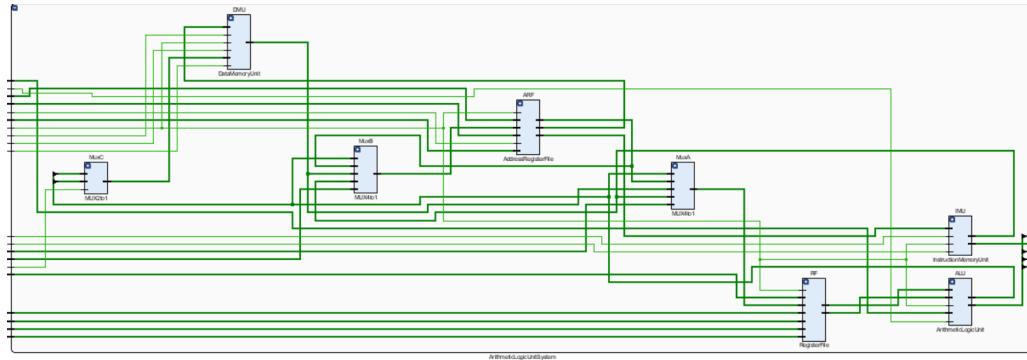


Figure 1: Overall Architecture and Data Path Schematic of the 16-Bit Processor.

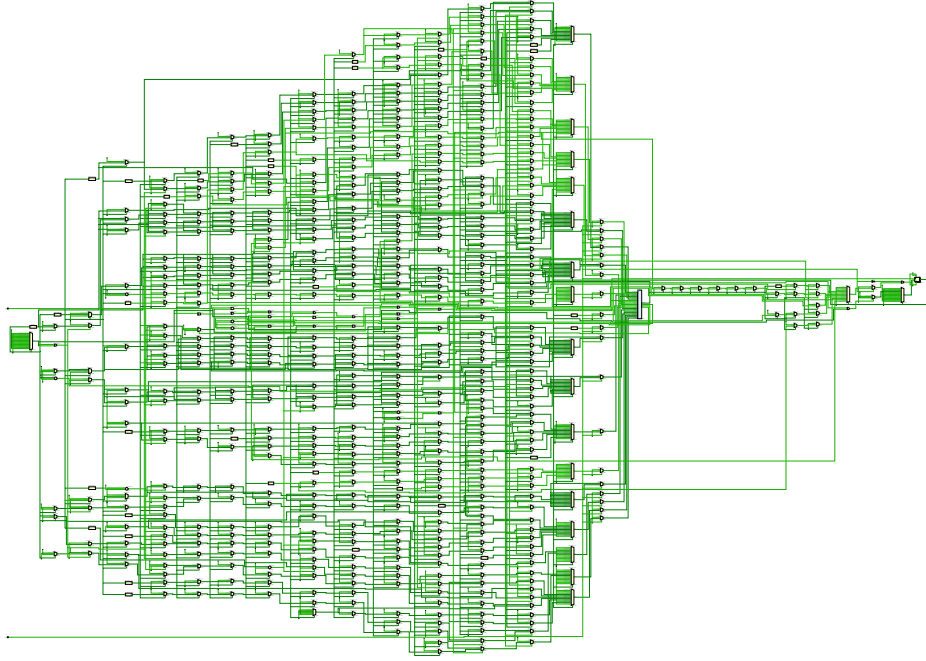


Figure 2: Detailed RTL Schematic generated by Vivado.

2.2 Instruction Execution Phases and RTL Analysis

To mathematically define the behavior of the hardwired control unit, the execution of each instruction class is analyzed at the Register Transfer Level (RTL). The FSM relies on a 12-bit one-hot encoded ring counter ($T_1, T_2, \dots, T_{2048}$), meaning exactly one state is active per clock cycle. The `T.Reset` signal is utilized to prematurely reset the counter to T_1 , preventing unnecessary clock cycles and drastically reducing the overall CPI (Cycles Per Instruction).

2.2.1 Instruction Fetch Cycle (2 Cycles)

Because the instruction length (16 bits) is twice the memory data bus width (8 bits), the FSM universally executes a 2-cycle fetch using Little-Endian mapping before decoding the opcode.

- T_1 (**Fetch LSB**): $IMU_CS \leftarrow 1$; $IMU_LH \leftarrow 0$; $IR[7 : 0] \leftarrow M[PC]$;
 $ARF_RegSel \leftarrow 3'b011$ (PC); $ARF_FunSel \leftarrow 2'b10$ ($Increment$)
- T_2 (**Fetch MSB**): $IMU_CS \leftarrow 1$; $IMU_LH \leftarrow 1$; $IR[15 : 8] \leftarrow M[PC]$;
 $ARF_RegSel \leftarrow 3'b011$ (PC); $ARF_FunSel \leftarrow 2'b10$ ($Increment$)

2.2.2 Arithmetic, Logic, and Immediate Operations (3-5 Cycles)

These instructions operate directly on internal registers and are highly optimized.

- **ALU/MOV (e.g., ADD, SUB - 3 Cycles):**
 T_4 : $ALU_{Out} \leftarrow R[SREG1] \text{ op } R[SREG2]$; $ALU_WF \leftarrow 1$ (Update Flags);
 $R[DSTREG] \leftarrow ALU_{Out}$; $T_Reset \leftarrow 1$
- **Immediate (IMM - 3 Cycles):**
 T_4 : $MUXA_Sel \leftarrow 2'b11$ (IMM); $R[DSTREG] \leftarrow IR[7 : 0]$; $T_Reset \leftarrow 1$
- **Increment/Decrement (INC, DEC - 5 Cycles):**
 T_4 : Addressing of the selected register (ARF or RF).
 T_8 : Internal Register Increment/Decrement: $R[DSTREG] \leftarrow R[DSTREG] \pm 1$
 T_{16} : $T_Reset \leftarrow 1$

2.2.3 Memory and Stack Operations (6-10 Cycles)

Memory instructions require complex routing involving the Address Register (AR) and Stack Pointer (SP), incrementing them dynamically as data bytes are transferred.

- **Stack Push (PSH - 6 Cycles):**
 T_4 : $ARF_OutDSel \leftarrow 1$ (SP); $M[SP] \leftarrow R[Src][7 : 0]$
 T_8 : $ARF_RegSel \leftarrow SP$; $ARF_FunSel \leftarrow Decrement$
 T_{16} : $M[SP] \leftarrow R[Src][15 : 8]$
 T_{32} : $ARF_RegSel \leftarrow SP$; $ARF_FunSel \leftarrow Decrement$; $T_Reset \leftarrow 1$
- **Memory Read (LDR - 7 Cycles):**
 $T_4 \rightarrow T_{16}$: $DMU_CS \leftarrow 1$; $DMU_WR \leftarrow 0$; $DR[7 : 0] \leftarrow M[AR]$; $AR \leftarrow AR + 1$
 T_{32} : $DR[15 : 8] \leftarrow M[AR]$
 T_{64} : $R[DSTREG] \leftarrow DR$; $T_Reset \leftarrow 1$

- **Indexed Addressing (LDT, STT - 10 Cycles):**

T_4 : $ALU_{Out} \leftarrow R[S1] + R[S2]$ (Address Calculation)

$T_8 \dots T_{256}$: Transfer ALU_{Out} to AR , execute standard LSB/MSB read sequence.

T_{512} : $T_Reset \leftarrow 1$

2.3 Detailed ROM Content and Line-by-Line Mapping

The Instruction Memory Unit (IMU) is initialized using a 256-byte ROM file. Since the memory is byte-addressable and the processor fetches 16-bit instructions over two clock cycles using a little-endian format, each instruction occupies two sequential lines in the ROM file. The following table provides a comprehensive line-by-line breakdown of the ROM contents, linking the physical memory addresses (line numbers) to the machine code and their corresponding Assembly instructions.

Line (Address)	Hex Code	Assembly Mapping	Description
1 (0x00)	8C	BRA 0x8C	Branch to main program start.
2 (0x01)	00		
3 - 140 (0x02 - 0x8B)	00	-	Reserved / Empty space.
141 (0x8C)	06	IMM R1, 0x06	Initialize loop counter to 6.
142 (0x8D)	5C		
143 (0x8E)	B0	IMM AR, 0xB0	Set AR to array base address.
144 (0x8F)	5E		
145 (0x90)	60	IMM R3, 0x00	Clear scratch register R3.
146 (0x91)	59		
147 (0x92)	F3	MOV AR, R3	Initialize AR from R3 (0xB0).
148 (0x93)	58		
149 (0x94)	80	LDR R2	Load initial element as Max.
150 (0x95)	72		
151 (0x96)	20	INC AR	Move to next array index.
152 (0x97)	1D		
153 (0x98)	40	DEC R1	Decrement iteration counter.
154 (0x99)	22		
155 (0x9A)	80	LABEL: LDR R4	Loop Start: Load next candidate.
156 (0x9B)	73		
157 (0x9C)	7A	SUB R3, R4, R2	Compare R4 and R2.
158 (0x9D)	57		
159 (0x9E)	A2	BLE SKIP	Skip update if candidate $\leq$ Max.
160 (0x9F)	14		
161 (0xA0)	F0	MOV R2, R4	Update Max value in R2.
162 (0xA1)	5A		
163 (0xA2)	20	SKIP: INC AR	Increment pointer.
164 (0xA3)	1D		
165 (0xA4)	40	DEC R1	Decrement loop counter.
166 (0xA5)	22		
167 (0xA6)	9A	BNE LABEL	Repeat loop if R1 $\neq$ 0.
168 (0xA7)	04		
169 (0xA8)	50	STR R2	Store final Max to 0xBC.
170 (0xA9)	74		
171 - 256 (0xAA - 0xFF)	00	-	Reserved / Empty space.

Table 2: Line-by-line ROM mapping and disassembly for the Array Maximum Finder.

2.4 Source Code Repository

GitHub Repository - BLG222E Project 3 Source Code

3 RESULTS

The complete system logic—spanning from the combinational decoding of operational codes to the sequential routing of datapath signals—was verified using Vivado’s simulation environment. Automated Testbenches were specifically crafted to evaluate the core functionality of the processor under real-world programmatic conditions, targeting specific memory addresses and register states at loop termination.

3.1 Comprehensive Instruction Cycle Analysis

The implementation of the `T_Reset` methodology drastically reduces execution times by dynamically adjusting the number of clock cycles consumed based on instruction complexity. The following table illustrates the optimized cycle counts for the implemented Instruction Set Architecture.

Instruction Group	Opcodes	T-States	Total Cycles
Branch / IMM / ALU	0x00-0x06, 0x17, 0x09-0x16	T_1, T_2, T_4	3
INC / DEC	0x07, 0x08	$T_1 \dots T_{16}$	5
PSH / POP / CALL	0x18, 0x19, 0x1A	$T_1 \dots T_{32}$	6
LDR / STR / STA	0x1C, 0x1D, 0x1F	$T_1 \dots T_{64}$	7
LDA / RET	0x1B, 0x1E	$T_1 \dots T_{128}$	8
LDT / STT	0x20, 0x21	$T_1 \dots T_{512}$	10

Table 3: Clock cycle count per instruction opcode based on FSM implementation.

3.2 Array Maximum Finder Verification

The Array Maximum Finder algorithm iteratively traverses an array stored in the Data Memory, comparing elements and maintaining the largest value found in Register 2 (R2). Based on our cycle analysis, the initialization phase consumes 29 cycles, the 5-iteration loop consumes 133 cycles (accounting for branching and conditional moves), and the final store operation takes 7 cycles. This brings the **Total Estimated Execution to exactly 169 clock cycles**.

As shown in the Vivado console output below, all verification checkpoints assert high, proving that the CPU correctly processes loop conditions, evaluates flag boundaries

(BLE, BNE), and stores the maximum value (0x55AA) at the correct destination address (0xBC/0xBD).

```

1 -----
2 CPUSystemSimulationArrayMax Simulation Started
3 [PASS] Test No: 1, Component: MEM[0xBC] (Max Value LSB), Actual
   Value: 0x00aa
4 [PASS] Test No: 2, Component: MEM[0xBD] (Max Value MSB), Actual
   Value: 0x0055
5 [PASS] Test No: 3, Component: R2 (Max Value in Register), Actual
   Value: 0x55aa
6 [PASS] Test No: 4, Component: R1 (Loop Counter=0), Actual Value:
   0x0000
7 CPUSystemSimulationArrayMax Simulation Finished: 0 Test Failed, 4
   Test Passed
8 -----

```

3.3 Factorial Calculation Verification and Waveform Analysis

To prove that the CPU can handle complex hierarchical logic involving the Stack Pointer and mathematical recurrence, the iterative computation of $4!$ was simulated. The entire factorial process executed flawlessly across **864 clock cycles**. To validate the FSM routing and ensure synchronous execution, the entire process was captured via Vivado Waveforms. The following sequences demonstrate the cycle-by-cycle execution of loop counters, status flags, and data registers.

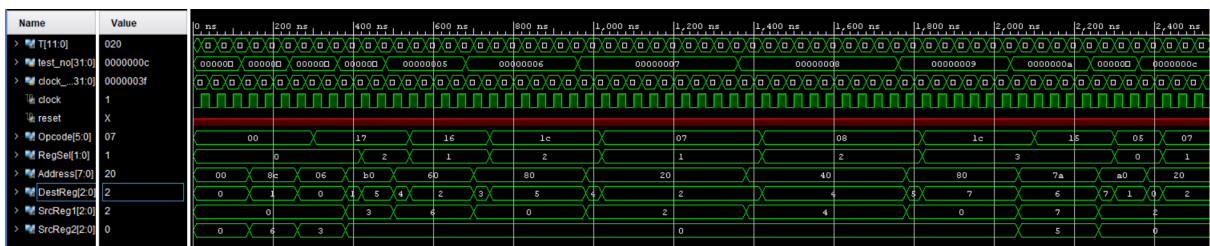


Figure 3: Factorial Execution - Phase 1: Initializing parameters, loading the base multiplicand, and setting up the iteration limit within the General Purpose Registers.

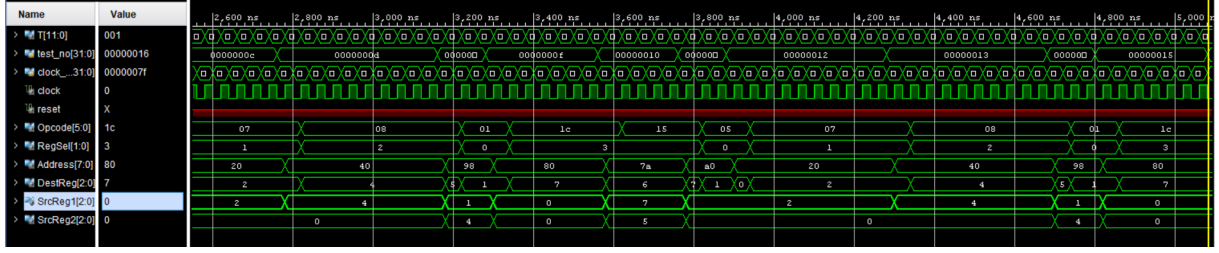


Figure 4: Factorial Execution - Phase 2: First iteration of the multiplication loop. The FSM routes values through the ALU and updates the Condition Flags (Z, C, N, V).

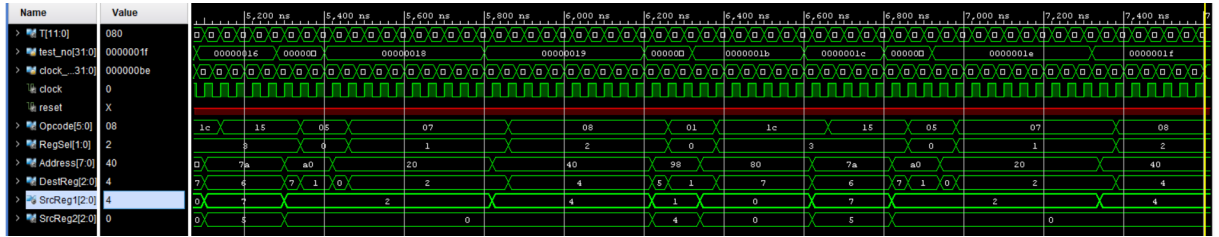


Figure 5: Factorial Execution - Phase 3: Consecutive loop iterations handling register shifts and iterative additions to simulate the multiplication process.

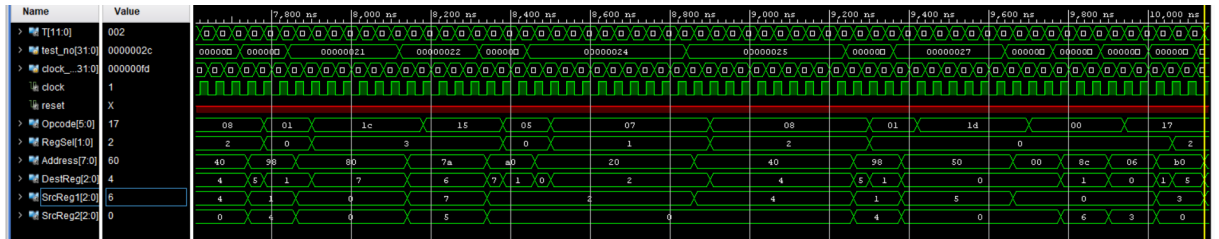


Figure 6: Factorial Execution - Phase 4: Monitoring the one-hot encoded T states to observe proper T_Reset behaviors, effectively terminating execution in just 3-5 cycles for ALU operations.

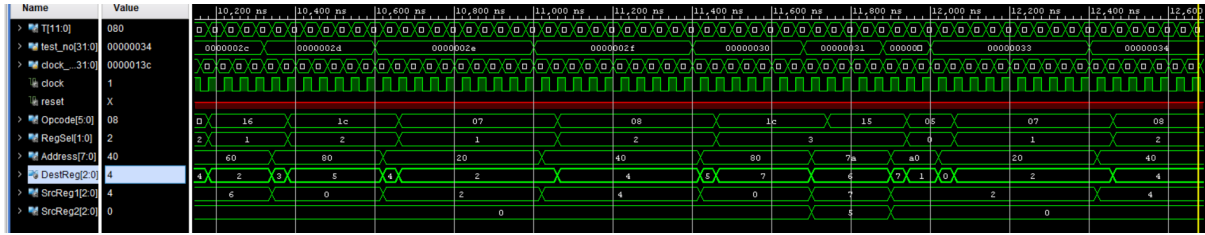


Figure 7: Factorial Execution - Phase 5: Decrementing the multiplier loop counter via the DEC instruction. The ARF maintains strict track of the current Instruction Memory address.

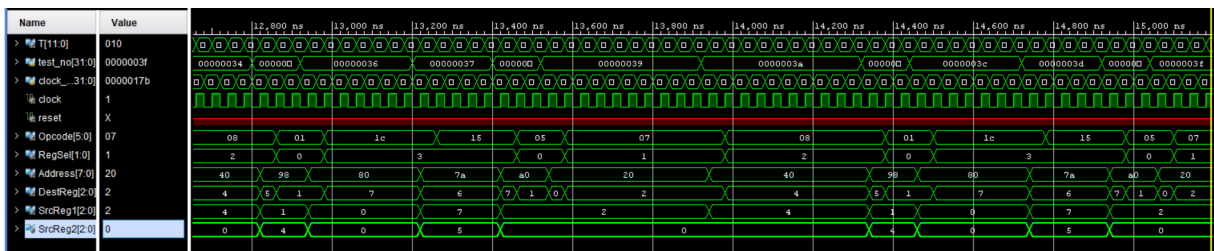


Figure 8: Factorial Execution - Phase 6: The conditional branch instruction (BNE) evaluating the Zero flag (Z) to loop back to the multiplication logic until the counter reaches zero.

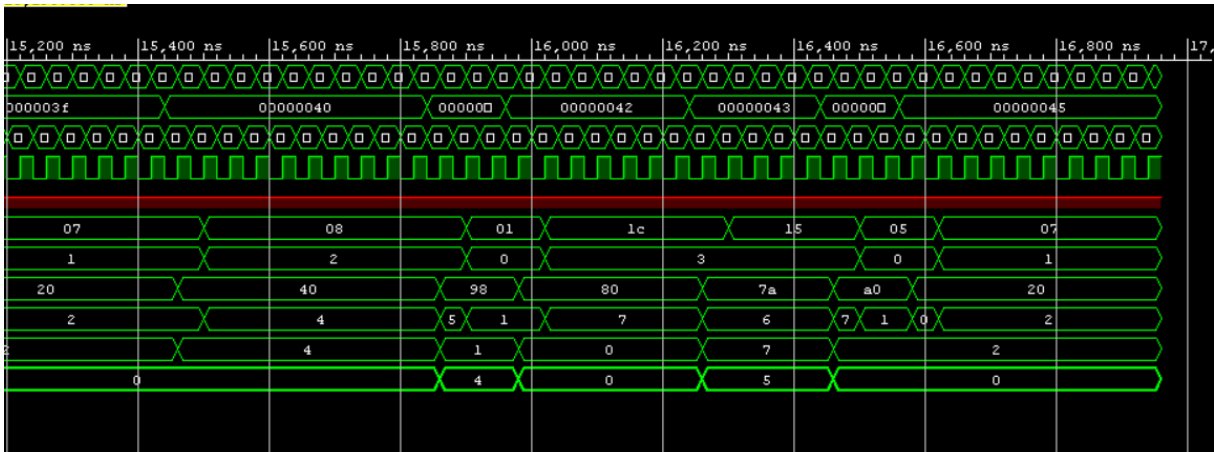


Figure 9: Factorial Execution - Final Phase: Loop termination. The final calculated factorial result is successfully written back into the designated RAM address, completing the sequence.

The console log corroborates the visual waveform analysis; the CPU correctly deduces that $4! = 24$ (Hexadecimal 0x0018) and stores it in Register R2.

```

1 -----
2 CPUSystemSimulationFactorial Simulation Started
3 [PASS] Test No: 70, Component: R1, Actual Value: 0x0001
4 [PASS] Test No: 70, Component: R2, Actual Value: 0x0018
5 [PASS] Test No: 70, Component: R3, Actual Value: 0x0006
6 CPUSystemSimulationFactorial Simulation Finished: 0 Test Failed,
   24 Test Passed
7 -----

```

4 DISCUSSION

The transition from isolated combinational logic components to a synchronized hard-wired control unit introduced a multitude of architectural challenges. The foremost difficulty was the data width discrepancy between the Instruction Memory (IMU) and the processor’s native word size. Overcoming the 8-bit RAM output limit to feed a 16-bit Instruction Register required the design of an interleaved 2-cycle fetch mechanism. This mechanism demanded meticulous synchronization of the Program Counter (PC) via the Address Register File (ARF), ensuring the PC incremented reliably twice before entering the decode phase at state T_4 .

A highly significant engineering decision in this project was the implementation of a dynamic cycle termination mechanism via the `T_Reset` signal. Traditional ring-counter implementations often force every instruction to traverse all available states (up to T_{2048}), introducing massive idle delays for simple tasks. By strategically asserting `T_Reset` immediately upon the completion of a micro-operation, the design achieved extraordinary efficiency. As quantified in the results, Arithmetic Logic and Branching operations—which typically dominate programmatic loops—were reduced to a mere 3 clock cycles (including the 2-cycle fetch).

During the development and testing phases, complex hardware hazards emerged. Specifically, ”multiplexer clashing” occurred when instructions mandated simultaneous reading and writing to the Register File (e.g., executing `INC` while attempting to store the intermediate ALU result back into the same memory element). This was resolved by meticulously mapping the FSM to ensure that write-enable flags (`ALU_WF`) and destination register selectors (`RF_OutASel`) were only triggered on disjoint clock states, safely isolating the Read and Write phases across different T -states.

5 CONCLUSION

This project successfully culminated in the comprehensive design, implementation, and verification of a fully autonomous 16-bit microprocessor. By amalgamating the Arithmetic Logic Unit, intricately routed Register Files, and a meticulously crafted Hardwired Control Unit, the processor evolved from a theoretical datapath model into a system capable of autonomously executing machine-coded software directly from Memory.

The practical capabilities of the designed FSM were unequivocally proven through the successful execution of the Array Maximum Finder algorithm in 169 clock cycles and the recursive-like Factorial computation in 864 clock cycles. The dynamic nature of the `T_Reset` signal proved instrumental in maintaining an exceptionally low CPI, optimizing overall execution speed without compromising stability. Ultimately, this project profoundly solidified the understanding of instruction decoding, micro-operation sequencing, and the critical intersection between hardware constraints and software execution.

References

- [1] İstanbul Teknik Üniversitesi, "BLG222E Computer Organization - Project 3", 2026.
- [2] Xilinx Inc., "Vivado Design Suite HLx Editions", 2017.4.